

Lesson 5 — Virtualization vs. Containerization & Docker Intro · Homework

Topics: VM vs. container isolation model, overcommit (CPU/RAM/swap/storage), stateful vs. stateless, leader/follower & quorum, Docker images vs. containers, `Dockerfile`, Docker Hub.

Part A — Virtualization vs. Containerization (theory)

Answer in `answers.md`, in your own words:

1. In 5–8 lines: why is a container's isolation weaker than a VM's? Name specifically what a container shares with the host that a VM does not.
2. CPU limit vs. memory limit on a container — what actually happens when each is exceeded, and why are they enforced differently?
3. Pick one app you've deployed so far in this course (nginx, MariaDB, your own site) and classify it stateful or stateless. Justify it in 2–3 lines.
4. A cluster has 4 nodes. Explain why that is **not** meaningfully safer than a 3-node cluster, using the split-brain/quorum math from class.
5. Why does a distributed system elect a single leader instead of letting every node accept writes independently? Name the specific problem this avoids.

Part B — Docker: build and run locally

1. Confirm Docker is installed: `docker --version`.
2. Write your own `Dockerfile` (do **not** just copy the class example) that: - starts `FROM nginx:alpine` (or another base image of your choice), - copies in **your own** HTML page (something more than one line — put your name and today's date in it), - exposes port 80.
3. Build it: `docker build -t <your-dockerhub-username>/homework5:v1`.
4. Run it and prove it works: `docker run -d -p 8080:80 <image>` then `curl localhost:8080`.
5. `docker exec -it <container> bash` (or `sh` on alpine) and run `cat /etc/os-release` — paste the output.

Part C — Docker Hub: push your image

1. Create a free account at hub.docker.com if you don't already have one.
2. Log in from your terminal: `docker login`.
3. Make sure your image is tagged with your Docker Hub username (Part B.3 already did this — `<username>/homework5:v1` — the username **must** match your Docker Hub account or the push will be

rejected).

4. Push it: `docker push <your-dockerhub-username>/homework5:v1`.
5. Prove it's really there: open your repository page on hub.docker.com in a browser and screenshot it, **or** from a different machine (or after `docker rmi` locally) run: `bash docker pull <your-dockerhub-username>/homework5:v1 docker run -d -p 8081:80 <your-dockerhub-username>/homework5:v1 curl localhost:8081` and paste the output showing your own page coming back down from Docker Hub, not from a local cache.

Deliverables

- `answers.md` with Part A's answers.
- Your `Dockerfile` and the HTML file it copies in.
- The `docker build`, `docker run`, `curl`, and `docker push` command output.
- A link to (or screenshot of) your public Docker Hub repository page.

Bonus

- Add a second stage/layer to your `Dockerfile` that also installs one extra tool (e.g. `curl`) inside the image, rebuild as `:v2`, push it, and explain in 2 lines why `:v1` and `:v2` are two separate tags of the *same* repository rather than two different images entirely.
- Run `docker history <your-image>` and explain what each layer corresponds to in your `Dockerfile`.